

الدليل الشامل لإدارة نافذة السياق وترشيد التوكنز في Antigravity

الدليل العملي لحوكمة الجلسات، كبح تدهور التركيز، وخفض خط الأساس في بيئة Antigravity IDE و CLI

🔑 الخلاصة التنفيذية والهدف المعماري

يقدم هذا المستند استراتيجيات هندسية متكاملة لإدارة نافذة السياق (Context Window) في Antigravity IDE والأداة الطرفية agcli، مع تصحيح المفاهيم الخاطئة حول 'أمان المليون توكن'، وتوضيح الركائز الأربع لحماية الذاكرة ومنع تشتت النماذج وتسريع الأداء البرمجي.

أولاً: تشريح المشكلة — وهم الأمان في 'المليون توكن' (The 1M Illusion)

يعتمد محرك Antigravity على نموذج Gemini 3.8 Flash (High) بنافذة سياق ضخمة تتسع لـ 1,048,576 توكن (1M Tokens). هذه السعة الهائلة (5 أضعاف Claude Code) توفر مساحة رحبة، لكنها توقع المطورين في فخين معماريين خطيرين:

1. ظاهرة تدهور التركيز وضياح التفاصيل (Context Rot): كلما تجاوزت الجلسة 400k إلى 500k توكن، تبدأ قدرة النموذج اللغوي على استرجاع التفاصيل الدقيقة بالانخفاض، مما يؤدي لنسيان القيود المعمارية الصارمة واللجوء لحلول سطحية وتكرار أخطاء برمجية سابقة.

2. تضخم الذاكرة العشوائية للمحرر (RAM & SQLite Bloat): يسجل المحرر كل خطوة واستدعاء أداة في قاعدة بيانات SQLite محلية (gemini/antigravity-ide/conversations/*.db/~). انتفاخ الجلسة يضخم قاعدة البيانات إلى أكثر من 40 ميغابايت ويرفع استهلاك الذاكرة العشوائية إلى 2.5GB، مما يبطئ واجهة المحرر ويسبب أخطاء الذاكرة الافتراضية للويندوز (errno=1455).

ثانياً: الحقيقة الهندسية — مضاعف استدعاء الأدوات (Tool Call Multiplier)

الخطر الأكبر في إدارة السياق ليس الوصول لسقف المليون، بل التكلفة التراكمية في كل دورة عمل:

• بنية استدعاء النماذج: في كل خطوة تفكير أو استدعاء لأداة برمجية (Tool Call)، يعيد المحرك إرسال كامل سياق المحادثة السابق من البداية.

• الحسبة الرياضية الصادمة: إذا كانت جلستك منتفخة بـ 200,000 توكن، ونفذ الوكيل 15 أداة برمجية لإنجاز ميزة معينة، فإن إجمالي التوكنز المعالجة والمحسوبة على كوتا الحساب هو: $3 = 15 \times 200,000$ ملايين توكن تم حرقها في مهمة واحدة! بينما إبقاء الجلسة عند 80,000 توكن يستهلك 1.2M توكن فقط، محققاً وفراً قدره 1.8 مليون توكن.

ثالثاً: الركيزة الأولى — خفض خط الأساس في Antigravity (The Baseline)

عند فتح أي جلسة جديدة، يبدأ مؤشر الاستهلاك فعلياً من 110,000 إلى 135,000 توكن (ما يعادل 11% إلى 13% من المليون) محجوزة للتعليمات الأساسية والأدوات والمهارات المحقونة. يوضح الجدول التالي المكونات المهدرة وكيفية خفضها:

المكون المهدر	الحجم التقريبي	الإجراء والتوصية الهندسية
سيرفرات MCP غير المستخدمة (مثل hostinger بـ +120 أداة)	25k - 35k توكن	تعطيلها من ملف mcp_config.json وحصرها في بيئات الويب فقط.
المهارات الفائضة (حزم الكيمياء والأحياء وورودبريس)	20k - 30k توكن	تشغيل خطاف التخصيص الذكي (Hook 3) لحذف المهارات الزائدة تلقائياً.
قواعد الدستور المتضخمة داخل AGENTS.md	15k - 20k توكن	عزل الدروس المستفادة وحصرها في MEMORY_STORE.md اللامركزي.
ملفات السياق المحقون غير المفلترة	10k - 15k توكن	تنقيح ACTIVE_CONTEXT_INJECTION.md وأرشفة القيود المنتهية.

الأثر المباشر: ينخفض خط الأساس فوراً من ~135k إلى نحو 65k-70k توكن، موفراً أكثر من 65,000 توكن في كل دورة طوال اليوم!

رابعاً: الركيزة الثانية — المقارنة المعمارية بين Antigravity IDE و CLI (agcli)

في إطار حوكمة أسطول الـ (Hook 22) CLIs، يمتلك Antigravity وجهين تشغيليين يجب الفصل التام بينهما:

وجه المقارنة	Antigravity IDE (Workbench)	Antigravity CLI (agcli run)
طبيعة العمل	بيئة التطوير الرسومية التفاعلية للمطور.	المنفذ الطرفي الذاتي غير التفاعلي (Headless).
الاستخدام الأمثل	التخطيط المعماري ومراجعة الـ Diffs والـ Artifacts.	بناء شاشات الـ UI، توليد الأكواد، وتحديث الذاكرة.
استهلاك الذاكرة	مرتفع (2.0GB – 2.5GB RAM) بسبب Electron.	منخفض جداً ومعزول ويحرر الذاكرة فور الانتهاء.
إدارة السياق	محادثة ممتدة تتراكم في قاعدة بيانات SQLite.	سياق مؤقت ينتهي مع انتهاء الأمر.
القاعدة المعمارية	يُحظر توجيه أوامر الـ CLI الآلية إليها منعاً للنسخ اليدوي.	المنفذ المعتمد للتفويض المباشر الذاتي من الأسطول.

خامساً: الركيزة الثالثة — عتبات الاستهلاك والتصفير في نافذة الـ 1M

بناءً على منحى تركيز النماذج، يتم تقسيم نافذة المليون إلى أربعة نطاقات تشغيلية دقيقة:

- النطاق الأخضر (0 – 300k توكن / 0% – 30%): منطقة الأداء الفائق (Sweet Spot) — أقصى درجات الذكاء والتركيز وانعدام تام للهلوسة. هذه هي المنطقة المثالية للتصميم الهيكلي وحل المعضلات المعمارية الكبرى.
- النطاق الأصفر (300k – 600k توكن / 30% – 60%): منطقة الإنتاجية المستقرة — استقرار عالي في الأداء مع ضرورة الالتزام بالقرءة المقتضبة للملفات وتجنب طباعة سجلات البناء.
- النطاق البرتقالي (600k – 800k توكن / 60% – 80%): منطقة التحذير — بداية تشتت التركيز. التوجيه الهندسي: إنهاء الميزة الحالية فوراً، وعمل Git Commit، وحفظ الذاكرة في MEMORY_STORE.md.
- النطاق الأحمر (< 800k توكن / < 80%): منطقة الخطر والتصفير الإلزامي — يُحظر بدء أي ميزة جديدة. يتم تطبيق خطاف التصفير الاستراتيجي (Hook 25) وتجميد الحالة في CURRENT_STATE.md وبدء جلسة جديدة نظيفة.

سادساً: الركيزة الرابعة — المراقبة اللحظية باستخدام أداة ag-context

تم ابتكار وبناء أداة طرفية متخصصة ومدمجة في النظام لقرءة استهلاك التوكنز لحظياً مباشرة من محرك Antigravity:

⚡ الأوامر التشغيلية اليومية لأداة ag-context

- عرض الحالة الفورية: ag-context
- شاشة المراقبة الحية في الوقت الفعلي (تحديث تلقائي كل 3 ثوانٍ): ag-context -w
- فحص تاريخ استهلاك التوكنز لآخر 5 خطوات: ag-context -H 5
- مخرجات JSON للتكامل مع السكريبتات والوكلاء: ag-context -j

سابعاً: الركيزة الخامسة — قواعد النظافة اليومية لكبح انتفاخ السياق

1. القراءة المجزأة للملفات (Slice Notation): يُحظر استدعاء `view_file` لقراءة ملفات كاملة تتجاوز 100 سطر. يجب دائماً استخدام معاملي `StartLine` و `EndLine` لقراءة المقطع المستهدف فقط.
2. الاستبدال الموضعي: اعتماد `replace_file_content` لتعديل الأسطر المحددة وتجنب `write_to_file` لإعادة كتابة ملفات كاملة.
3. حظر طباعة مخرجات البناء الخام: مخرجات Gradle أو NPM تلتهم من 10k إلى 30k توكن عند طباعتها في الكونسول. القاعدة: توجيه المخرجات لملف خارجي (`gradlew ... > build.log/.`) وقراءة سطور الخطأ فقط.
4. تجميع الطلبات (Batching): تجميع الاستفسارات والمهام في رسالة واحدة منسقة يوفر دورات كاملة من إعادة إرسال سياق المحادثة السابق.

ثامناً: الخلاصة التنفيذية وقائمة الإجراءات الفورية

1. تعطيل خوادم MCP الزائدة في `mcp_config.json` (يوفر 25k-35k توكن).
2. تشغيل خطاف التخصيص الذكي (Hook 3) لحذف المهارات غير المستخدمة (يوفر 20k-30k توكن).
3. المتابعة اليومية لنسبة الامتلاء عبر أمر `ag-context`.
4. توجيه مخرجات البناء للملفات لتفادي قفزات التوكيز المفاجئة.
5. التصفير الاستراتيجي عند 60% إلى 70% للبقاء دائماً في منطقة الأداء الفائق (Sweet Spot).